WOUNDMAP AI

Antigravity Autonomous Execution Blueprint

Multi-phase agent workflow • GitHub • Google Colab • Research-grade controls

Objective	Use Antigravity as the orchestration and coding agent to build, test, document and package the WoundMap AI research prototype in controlled phases.
Outputs	Audited dataset pipeline, reproducible segmentation experiments, evaluation package, Streamlit demo, report and presentation assets.
Operating principle	Automate engineering work, not scientific judgement. The agent must stop at gates when data, labels, permissions or research decisions are uncertain.
Integrations	GitHub for code/version control; Colab for notebook execution and compute; Drive for large data/checkpoints only if supported and approved.

1. What Antigravity should do

Antigravity is the project orchestrator and implementation agent. It should inspect the repository, plan work, edit files, run tests where execution is available, use GitHub for controlled commits, and prepare Colab notebooks/scripts. It must not silently decide scientific assumptions, fabricate results, bypass permissions, or claim a notebook ran when it only generated code.

Role separation

• **Antigravity:** planning, repository changes, code generation, tests, documentation, issue tracking and phase hand-offs.

• **Google Colab:** notebook execution and GPU/CPU training. Direct remote execution depends on an explicitly configured, supported integration; do not assume native Colab control.

• **GitHub:** source of truth for code, notebooks, branches, commits and pull requests.

• **Google Drive:** suitable for raw dataset archives, large checkpoints and outputs if a permitted integration is configured. Do not commit raw data or large checkpoints to GitHub.

• **Researcher:** approves label interpretation, split strategy, experiment design, use of clinical data, metrics and scientific claims.

Integration boundary: authorising a Google account does not prove Antigravity can operate every Colab UI control or access every Drive file. First test the exact available actions. If Colab execution is unsupported, the agent must prepare a ready-to-run notebook and a manual execution checklist; you run it and return logs/artifacts.

2. Repository structure

```
WoundMapAI/
■■■ AGENTS.md # persistent agent rules
■■■ README.md # setup, project and reproduction guide
■■■ PLAN.md # phases, status, gates and decisions
■■■ CHANGELOG.md # dated change history
■■■ requirements.txt
■■■ .gitignore # raw data, secrets, checkpoints, caches
■■■ configs/
■ ■■■ dataset.yaml
■ ■■■ split.yaml
■ ■■■ baseline.yaml
■ ■■■ ablations.yaml
■■■ data/
■ ■■■ README.md # source, licence, expected layout
■ ■■■ manifests/ # metadata CSV only
■■■ notebooks/
■ ■■■ 00_dataset_audit.ipynb
■ ■■■ 01_preprocessing_eda.ipynb
■ ■■■ 02_baseline_training.ipynb
■ ■■■ 03_ablation_experiments.ipynb
■ ■■■ 04_evaluation_error_analysis.ipynb
■■■ src/woundmap/
■ ■■■ data.py
■ ■■■ masks.py
■ ■■■ preprocessing.py
■ ■■■ models.py
■ ■■■ train.py
■ ■■■ metrics.py
■ ■■■ inference.py
■■■ tests/
■ ■■■ test_mask_mapping.py
■ ■■■ test_pairing.py
■ ■■■ test_transforms.py
■ ■■■ test_metrics.py
■■■ app/streamlit_app.py
■■■ outputs/ # git-ignored
■■■ reports/
■ ■■■ literature_review.md
■ ■■■ experiment_protocol.md
■ ■■■ results.md
■ ■■■ limitations.md
■■■ presentation/
```

The agent may simplify this structure initially. Do not spend time building packaging infrastructure before dataset validity and the baseline experiment are established.

3. Persistent agent rules — AGENTS.md

```
# WoundMap AI — Agent Rules

MISSION
Build a reproducible research prototype for multi-class wound-tissue
```

```
semantic segmentation and controlled colour/illumination experiments.

SCIENTIFIC SAFEGUARDS
1. Never invent dataset structure, image-mask pairs, class IDs, mask colours,
patient IDs, paper findings, metrics or results.
2. Inspect actual files; cite the evidence for decisions.
3. Stop if pairing, annotation meaning, licence or label mapping is unclear.
4. Preserve raw data; never overwrite originals.
5. Split by patient/wound group when reliable group IDs exist. Prevent
image/patch leakage. If IDs are absent, disclose this and seek approval.
6. Split before augmentation. Apply geometric transforms identically to
image and mask. Use nearest-neighbour interpolation for categorical masks.
7. Keep test data untouched during model selection.
8. Report per-class Dice/IoU, support and absent-class handling.
9. Do not claim clinical validity, diagnosis, perfusion, oxygenation or
tissue chemistry from ordinary RGB photographs.
10. Distinguish generated code, locally tested code, Colab-executed code
and independently reproduced results.

ENGINEERING SAFEGUARDS
- Work on a feature branch; make small, reviewable commits.
- Ask before destructive actions, expensive jobs, external uploads or publishing.
- Never delete, overwrite, force-push, change visibility or expose credentials
without explicit approval.
- Never commit tokens, .env files, raw patient data or large checkpoints.
- Add tests for pairing, mask mapping, transforms and metric calculations.
- Record environment, package versions, seeds, configs and run identifiers.
- Keep code modular and explainable to an undergraduate researcher.

PHASE PROTOCOL
For every phase: inspect -> plan -> implement -> test -> show evidence ->
summarise changes -> request approval at the gate.
Do not start the next gated phase until approval is received.
```

Autonomy permissions

• **Routine actions:** inspect files, create new files, edit code on working branch, run non-destructive tests, generate reports.

• **Ask first:** install unusual dependencies, download large datasets, access external services, run expensive GPU jobs, upload data, change repository settings, publish or deploy.

• **Never:** commit secrets, bypass access controls, use unconsented clinical data, fabricate metrics or make destructive changes without approval.

4. Phase map and acceptance gates

Phase	Agent work	Deliverables	Human gate
0 Bootstrap	Inspect tools; create repo structure, AGENTS.md, PLAN.md, README, .gitignore and dependency file.	Private repo, branch, setup report.	Confirm repo and integration capabilities.
1 Dataset audit	Inspect WoundTissue tree, image-mask pairs, dimensions, mask colours, formats, licence and grouping metadata.	Audit notebook, manifest, contact sheet, audit report.	Verify pairing and class mapping.
2 Research protocol	Review dataset paper and related work; define hypotheses, split, metrics and ablation matrix.	Literature notes and approved protocol.	Supervisor/user approves protocol.
3 Preprocessing	Implement deterministic resize, colour conversion, optional illumination normalisation and augmentation.	EDA notebook, transform tests and figures.	Inspect overlays and mask integrity.
4 Baseline model	Implement U-Net/FPN candidate, verified output classes, loss, training, checkpointing and logs.	Training notebook, checkpoint and curves.	Sanity tests and predictions pass.
5 Evaluation	Compute class-wise metrics, confusion/error analysis and inference time.	Metrics CSV/JSON and figures.	Confirm test set was not used for tuning.
6 Ablations	Run controlled RGB/HSV/CIELAB and selected illumination condition.	Configs, logs and comparison table.	Verify runs and fair comparison.
7 Demo	Build Streamlit upload/inference/overlay/area summary with non-clinical warning.	App, test cases and screenshots.	Test fresh and invalid inputs.
8 Report	Prepare report, slides, README, reproducibility package and handover.	Final research package.	Verify every number and claim.

Phase 0 — Bootstrap and integration validation

Deliverables: Private repo, feature branch, agent rules, plan, integration capability report.

```
You are the autonomous engineering lead for WoundMap AI. Inspect this workspace and the available GitHub/Google integrations. Do not assume an integration exists just because I authorised an account connection. Report actual capabilities: repository read/write, branch/commit/PR, notebook file creation, Colab execution, Drive access and artifact retrieval.

If supported, create a new private GitHub repository named WoundMapAI or use the repository I explicitly select. Do not change visibility or overwrite anything without asking. Create the agreed folder structure, AGENTS.md, PLAN.md, README.md, CHANGELOG.md, requirements.txt and .gitignore. Create a feature branch and test permitted GitHub actions. Determine whether Colab can truly be executed remotely; if not, document notebook hand-off. Do not download data or train. Show evidence and request Phase 0 approval.
```

Phase 1 — Dataset acquisition and audit

Deliverables: Audit notebook, verified manifest, contact sheet, class mapping, licence record, unresolved issues.

```
Proceed with Phase 1 only. Obtain WoundTissue from its official repository if access and licence permit. Preserve the original archive and record source URL, access date, licence and checksum. Do not place raw images in GitHub. Inspect actual folders, filenames, dimensions, formats, unique mask RGB values and annotation formats. Verify pairs from evidence, not guessed filename substitutions. Produce 00_dataset_audit.ipynb, manifest CSV, contact sheet, class pixel-count report and data README. Investigate patient/wound group IDs. Stop on ambiguity. Do not train. Show at least 10 image/mask examples for human review and request approval.
```

Phase 2 — Literature and experiment protocol

Deliverables: Literature review, hypotheses, split decision, experiment protocol and approved config.

```
Use only verified dataset facts. Read the dataset paper and comparable wound-tissue segmentation research. Link traceable sources. Draft the research question: how do RGB, HSV and CIELAB representations, with and without one selected illumination-normalisation method, affect wound-tissue segmentation robustness? Define hypotheses, variables, baseline, ablation matrix, split policy, metrics, compute budget and limitations. Do not claim novelty without comparison to prior work. Save literature_review.md and experiment_protocol.md. Request supervisor/user approval before training.
```

Phase 3 — Preprocessing and EDA

Deliverables: Preprocessing code, notebook, tests, EDA plots and overlays.

```
After Phase 1 and 2 approval, implement preprocessing and 01_preprocessing_eda.ipynb. Use exact verified label mapping. Keep image/mask aligned; categorical masks use nearest-neighbour resizing. Apply geometric augmentation identically to both; colour transforms to image only. Implement RGB/HSV/CIELAB conversion with documented scaling. Add one selected illumination-normalisation method as an option. Add tests for valid IDs, alignment, shape and deterministic behaviour. Generate EDA plots and overlays. Do not train yet. Run tests and show visual evidence.
```

Phase 4 — Baseline segmentation

Deliverables: Training notebook/script, configs, logs, checkpoint, curves and prediction overlays.

```
After preprocessing tests pass, implement a simple U-Net or FPN. Use a pretrained encoder only if library compatibility and compute allow. Use the verified class count and approved split manifest; never create a new random split in training code. Include fixed seeds, configurable resolution/batch/epochs, suitable segmentation loss, early stopping, checkpointing and logs. Add one-batch overfit sanity test and tensor/class-range assertions. Prefer Colab GPU if genuinely available; otherwise provide CPU smoke test and manual Colab hand-off. Never claim GPU execution without logs. Ask before long jobs.
```

Phase 5 — Evaluation and error analysis

Deliverables: Evaluation notebook, metric files, confusion matrix, visual panels and error analysis.

```
Use selected checkpoint and untouched approved test manifest. Compute per-class Dice, IoU, precision, recall, support and macro averages. Define absent-class handling. Validate metric functions on a toy example. Generate confusion matrix and panels of input, ground truth, prediction and overlay. Measure inference time with warm-up and repeated runs, reporting hardware and resolution. Identify class confusions and boundary errors. Do not tune on test results. Save CSV/JSON and figures; state small-data limitations.
```

Phase 6 — Controlled ablations

Deliverables: Run configs, logs, per-class metrics, comparison table and interpretation.

```
After baseline evaluation is accepted, run approved RGB, HSV, CIELAB and selected illumination-normalisation conditions. Reuse identical splits, architecture, training budget, augmentation, optimiser, loss, evaluation code and seeds. Log every config and checkpoint. If compute is limited, prioritise baseline plus one alternative and mark unrun conditions. Compare class-wise metrics and absolute differences. Do not claim statistical significance from single runs. Save comparison table and distinguish observation from interpretation.
```

Phase 7 — Streamlit research demo

Deliverables: Working local app, tests, screenshots and run instructions.

```
Use selected checkpoint to build a minimal Streamlit app. Accept supported image, validate file/dimensions, apply exact inference preprocessing, show original and colour-coded overlay, report pixel proportions with denominator stated, and offer CSV export. Include model/version metadata and warning: research prototype only; not for diagnosis or treatment decisions. Do not give diagnosis, severity or treatment advice. Test at least five valid and invalid inputs. Document local run. Do not deploy publicly without approval.
```

Phase 8 — Report and handover

Deliverables: Report, slides, reproducibility README and final handover.

```
Use only verified notebook outputs, metrics, configs and cited papers. Draft technical report and 6–8 minute
presentation. Do not invent results, citations, significance or clinical claims. Include dataset/licence, annotation
audit, split, methods, experiment matrix, class-wise results, failure cases and limitations. State whether
patient-level grouping was possible. Create README reproduction instructions and handover notes: repository tree,
environment, Colab steps, artifacts, known issues and next actions. Ask before publishing or making repository public.
```

5. Colab + GitHub operating workflow

• Antigravity creates/updates notebook files in the GitHub feature branch.

• Open the notebook in Colab from GitHub (or copy it to Drive if required).

• Load the dataset into Colab from an approved Drive location or approved download source; raw data stays out of GitHub.

• Colab runs audit/training/evaluation and saves logs, metrics, figures and checkpoints to Drive or controlled artifact storage.

• Only small, appropriate outputs (configs, metrics summaries, selected figures) are committed; large checkpoints remain in Drive.

• Antigravity updates reports only after it receives actual logs and output files.

Fallback: if no direct Colab connector exists, Antigravity creates a top-to-bottom runnable notebook and an execution checklist. You click Run all in Colab, then provide the resulting logs/artifacts. Do not use fragile browser automation as proof of execution.

6. GitHub workflow and data safety

• Use main for stable work and phase branches such as phase/01-dataset-audit.

• Commit after each accepted phase; use concise messages and review diffs.

• Keep .env, tokens, raw images, patient data and large checkpoints out of Git.

• Never force-push, delete branches, publish releases or change repo visibility without approval.

• Use Issues or PLAN.md to track blockers, status, owner and acceptance criteria.

• Check dataset licence before redistribution, public demos or commercial use.

7. Mandatory automated tests

Test	Required check	On failure
Pairing	Every manifest pair exists; transformed dimensions align.	Stop; report unmatched files.
Mask mapping	Every pixel colour maps to a known ID.	Stop training; resolve mapping.
Transforms	Categorical labels remain discrete; geometry aligned.	Reject transform pipeline.
Split integrity	No duplicate image/patch/group leakage.	Rebuild under approved policy.
Model output	Channels, shapes and IDs match config.	Stop training.
Metrics	Dice/IoU pass hand-calculated toy test.	Do not publish metrics.
Inference	Checkpoint reloads; output dimensions valid.	Do not ship demo.

8. Project control dashboard

```
| Phase | Status | Evidence link | Blocker | Approval |
|---|---|---|---|---|
| 0 Bootstrap | NOT STARTED | | | |
| 1 Dataset audit | NOT STARTED | | | |
| 2 Research protocol | NOT STARTED | | | |
| 3 Preprocessing | NOT STARTED | | | |
| 4 Baseline model | NOT STARTED | | | |
| 5 Evaluation | NOT STARTED | | | |
| 6 Ablations | NOT STARTED | | | |
| 7 Demo | NOT STARTED | | | |
| 8 Report/handover | NOT STARTED | | | |
```

Antigravity must update PLAN.md only when evidence exists. Every phase hand-off must state: files changed; commands/actions run; test evidence; unresolved questions; risks; and exact approval required.

9. Master prompt — paste into Antigravity first

This starts Phase 0 only. Attach this PDF to the Antigravity workspace and paste the following instruction.

```
I am building WoundMap AI, an undergraduate biophysics/biomedical computer-vision research prototype for wound-tissue semantic segmentation. Use the attached Antigravity Autonomous Execution Blueprint as the governing specification.

Act as project orchestrator and senior ML engineer. Work in small phases with explicit acceptance gates. You may inspect the workspace, create files, edit code on a feature branch and run safe tests. Stop for approval before destructive actions, expensive jobs, external uploads, public publishing, changing repository visibility or making unresolved scientific decisions.

Do Phase 0 only:
1. Inspect workspace and report available tools/integrations.
2. Verify exactly which GitHub actions are available.
3. Verify whether Colab can actually be executed remotely or notebooks can only be created/opened.
4. Ask me to select/confirm the GitHub repository if not specified.
5. Create AGENTS.md, PLAN.md, README.md, CHANGELOG.md, requirements.txt, .gitignore and agreed folder structure.
6. Create a feature branch; do not push secrets or raw dataset files.
7. Validate setup and show repository tree and evidence.

Do not download data, train a model or start Phase 1 until I approve Phase 0. Never invent outputs or claim success without tool evidence. At hand-off report files changed, actions run, test evidence, unresolved questions, risks and exact approval needed.
```

10. Recommended initial checklist

- Confirm Antigravity version and actual GitHub/Google connector capabilities.
- Create or select a private GitHub repository; grant only the permissions required.
- Keep raw dataset and large checkpoints outside GitHub.
- Validate whether Colab execution is integrated or use notebook hand-off.
- Attach this PDF and paste the master prompt; approve Phase 0 only.
- Manually review the dataset audit before approving model development.
- Approve each later phase only when deliverables and tests are visible.

Target: high automation with traceability. Antigravity can handle repetitive engineering work; the researcher remains responsible for validating data, methodology, interpretation and scientific claims.